

CSC311 Comprehensive Study Notes

Machine Learning: Models, Losses, Optimization, Generalization, and Derivations

Prepared as a full-course revision textbook

Contents

1	The Big Picture of Machine Learning	5
1.1	Supervised Learning	5
1.2	Loss and Risk	5
2	Linear Regression	6
2.1	Model	6
2.2	Squared Error Objective	6
2.3	Gradient and Normal Equations	7
2.4	Geometric Interpretation	7
2.5	Probabilistic Interpretation	7
3	Regularization	8
3.1	Motivation	8
3.2	Ridge Regression	8
3.3	Lasso Regression	8
3.4	Bayesian View of Regularization	9
4	Bias-Variance Tradeoff	9
5	Binary Classification and Logistic Regression	10
5.1	Linear Classifiers	10
5.2	Sigmoid and Logistic Regression	10
5.3	Binary Cross-Entropy	10
5.4	Gradient Derivation	10
6	Multiclass Classification	11

6.1	Linear Scores	11
6.2	Softmax	11
6.3	Multiclass Cross-Entropy	12
6.4	Softmax Cross-Entropy Gradient	12
7	Optimization	12
7.1	Gradient Descent	12
7.2	Batch, Stochastic, and Mini-Batch Gradient Descent	12
7.3	Momentum	13
7.4	Convexity	13
8	Neural Networks	13
8.1	Motivation	13
8.2	Activation Functions	14
8.3	Forward Pass	14
8.4	Backpropagation	14
8.5	Output-Layer Error Signals	15
9	Generalization and Model Selection	15
9.1	Train, Validation, and Test Sets	15
9.2	Cross-Validation	15
9.3	Underfitting and Overfitting	15
10	Probabilistic Machine Learning	15
10.1	Maximum Likelihood Estimation	16
10.2	Maximum A Posteriori Estimation	16
10.3	Generative vs Discriminative Models	16
11	Naive Bayes	17
11.1	Conditional Independence Assumption	17
11.2	Bernoulli Naive Bayes	17
11.3	Multinomial Naive Bayes	17
12	Gaussian Discriminant Analysis	18

12.1 Model	18
12.2 Linear Discriminant Analysis	18
12.3 Quadratic Discriminant Analysis	18
13 Kernel Methods	18
13.1 Feature Maps	18
13.2 The Kernel Trick	19
13.3 Kernel Ridge Regression	19
14 Support Vector Machines	19
14.1 Maximum Margin Classifier	19
14.2 Soft-Margin SVM and Hinge Loss	20
15 Unsupervised Learning	20
15.1 K-Means Clustering	20
15.2 Gaussian Mixture Models	21
15.3 EM Algorithm	21
16 Principal Component Analysis	21
16.1 Variance Maximization View	21
16.2 Multiple Components and Reconstruction	22
17 Evaluation Metrics	22
17.1 Regression	22
17.2 Classification	22
17.3 ROC and AUC	23
18 High-Yield Derivations	23
18.1 Least Squares Normal Equations	23
18.2 Ridge Closed Form	23
18.3 Logistic Regression Gradient	24
18.4 Softmax Cross-Entropy Gradient	24
18.5 Backpropagation Skeleton	24
19 Exam-Ready Concept Comparisons	24

19.1 Loss vs Likelihood	24
19.2 Linear vs Nonlinear Models	25
19.3 Closed Form vs Iterative Optimization	25
19.4 Generative vs Discriminative	25
19.5 Why Validation Data Matters	25
20 Compact Formula Sheet	25
21 Recommended Relearning Order	26

1 The Big Picture of Machine Learning

Machine learning studies algorithms that improve performance on a task using data. In supervised learning, the dataset is

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where x_i is an input, y_i is a target, and N is the number of training examples. The goal is to learn a function $f_\theta(x)$, parameterized by θ , that predicts y well for new, unseen inputs.

A standard machine learning pipeline has four central ingredients:

1. **Model:** choose a family of functions f_θ .
2. **Loss:** define how bad a prediction is.
3. **Optimization:** minimize the training objective.
4. **Generalization:** evaluate whether the model performs well on unseen data.

The most important CSC311 mindset is:

$$\text{Model} + \text{Loss} + \text{Optimization} + \text{Generalization}.$$

Almost every topic in the course fits into this framework.

1.1 Supervised Learning

In supervised learning, every example has an input and a label. If $x_i \in \mathbb{R}^D$, then the design matrix is

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_N^\top & - \end{bmatrix} \in \mathbb{R}^{N \times D},$$

and the target vector for regression is $y \in \mathbb{R}^N$.

Regression uses real-valued labels:

$$y_i \in \mathbb{R}.$$

Binary classification uses two labels:

$$y_i \in \{0, 1\} \quad \text{or} \quad y_i \in \{-1, +1\}.$$

Multiclass classification uses:

$$y_i \in \{1, 2, \dots, K\}.$$

1.2 Loss and Risk

A loss function measures prediction error:

$$\ell(f_\theta(x_i), y_i).$$

The empirical risk is the average training loss:

$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

Training often means empirical risk minimization:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

The true population risk is

$$R(\theta) = \mathbb{E}_{(x,y) \sim p_{\text{data}}} [\ell(f_{\theta}(x), y)].$$

The central challenge is that we minimize $\hat{R}(\theta)$ but care about $R(\theta)$.

2 Linear Regression

2.1 Model

Linear regression predicts

$$\hat{y}_i = w^{\top} x_i + b.$$

By adding a constant feature, we can absorb the bias term into the parameter vector. Define

$$\tilde{x}_i = \begin{bmatrix} 1 \\ x_i \end{bmatrix}, \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}.$$

Then

$$\hat{y}_i = \theta^{\top} \tilde{x}_i.$$

In matrix form,

$$\hat{y} = X\theta.$$

2.2 Squared Error Objective

The least squares objective is

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^N (\theta^{\top} x_i - y_i)^2.$$

In matrix form:

$$\mathcal{L}(\theta) = \frac{1}{2} \|X\theta - y\|_2^2.$$

Expanding:

$$\mathcal{L}(\theta) = \frac{1}{2} (X\theta - y)^{\top} (X\theta - y) = \frac{1}{2} (\theta^{\top} X^{\top} X \theta - 2y^{\top} X \theta + y^{\top} y).$$

2.3 Gradient and Normal Equations

Using matrix calculus,

$$\nabla_{\theta} \mathcal{L}(\theta) = X^{\top}(X\theta - y).$$

At the optimum, set the gradient equal to zero:

$$X^{\top}(X\theta - y) = 0.$$

Therefore,

$$X^{\top}X\theta = X^{\top}y.$$

These are the normal equations. If $X^{\top}X$ is invertible,

$$\hat{\theta} = (X^{\top}X)^{-1}X^{\top}y.$$

2.4 Geometric Interpretation

The vector $\hat{y} = X\hat{\theta}$ is the projection of y onto the column space of X . The residual vector

$$r = y - \hat{y}$$

is orthogonal to every column of X because

$$X^{\top}(y - X\hat{\theta}) = 0.$$

So least squares finds the point in the span of the features closest to y .

2.5 Probabilistic Interpretation

Assume

$$y_i = w^{\top}x_i + b + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Then

$$p(y_i | x_i, w, b) = \mathcal{N}(y_i; w^{\top}x_i + b, \sigma^2).$$

The likelihood is

$$p(y | X, \theta) = \prod_{i=1}^N \mathcal{N}(y_i; \theta^{\top}x_i, \sigma^2).$$

Taking the negative log-likelihood gives

$$-\log p(y | X, \theta) = \frac{N}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \theta^{\top}x_i)^2.$$

Since the first term is constant in θ , maximizing likelihood is equivalent to minimizing squared error.

3 Regularization

3.1 Motivation

Overfitting happens when a model fits training data too closely and performs poorly on new data. Regularization adds a penalty to discourage overly complex parameters:

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda\Omega(\theta),$$

where $\lambda \geq 0$ controls regularization strength.

Large λ gives simpler models with higher bias and lower variance. Small λ gives more flexible models with lower bias and higher variance.

3.2 Ridge Regression

Ridge regression uses an L_2 penalty:

$$\Omega(w) = \frac{1}{2} \|w\|_2^2.$$

The objective is

$$\mathcal{L}(w) = \frac{1}{2} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2.$$

Gradient:

$$\nabla_w \mathcal{L}(w) = X^\top(Xw - y) + \lambda w.$$

Set to zero:

$$X^\top Xw - X^\top y + \lambda w = 0.$$

Therefore,

$$(X^\top X + \lambda I)w = X^\top y,$$

and

$$\boxed{\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y.}$$

Ridge shrinks weights toward zero, but usually does not set them exactly to zero.

3.3 Lasso Regression

Lasso uses an L_1 penalty:

$$\Omega(w) = \|w\|_1 = \sum_{j=1}^D |w_j|.$$

The objective is

$$\mathcal{L}(w) = \frac{1}{2} \|Xw - y\|_2^2 + \lambda \|w\|_1.$$

Lasso encourages sparsity, meaning many weights become exactly zero. This makes it useful for feature selection. Unlike ridge, lasso does not generally have a simple closed-form solution because $|w_j|$ is not differentiable at $w_j = 0$.

3.4 Bayesian View of Regularization

Regularization can be interpreted as a prior. MAP estimation solves

$$\hat{w}_{\text{MAP}} = \arg \max_w p(w \mid X, y).$$

By Bayes' rule,

$$p(w \mid X, y) \propto p(y \mid X, w)p(w).$$

Taking negative logs:

$$-\log p(w \mid X, y) = -\log p(y \mid X, w) - \log p(w) + C.$$

The likelihood term becomes the data loss, and the prior becomes the regularizer.

For ridge, the prior is Gaussian:

$$w \sim \mathcal{N}(0, \tau^2 I).$$

For lasso, the prior is Laplace:

$$p(w_j) \propto \exp(-\alpha |w_j|).$$

4 Bias-Variance Tradeoff

Assume

$$y = f(x) + \epsilon, \quad \mathbb{E}[\epsilon] = 0, \quad \text{Var}(\epsilon) = \sigma^2.$$

For squared error, the expected prediction error decomposes as

$$\mathbb{E}[(y - \hat{f}(x))^2] = \left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma^2.$$

Thus:

$$\text{Expected error} = \text{Bias}^2 + \text{Variance} + \text{Noise}.$$

Bias is systematic error. High bias means the model is too simple. Variance is sensitivity to the training set. High variance means the learned model changes greatly depending on the sample.

Situation	Bias	Variance
Underfitting	High	Low
Overfitting	Low	High
Good fit	Moderate	Moderate

Regularization usually increases bias but decreases variance.

5 Binary Classification and Logistic Regression

5.1 Linear Classifiers

A binary linear classifier computes

$$z = w^\top x + b.$$

For labels $y \in \{0, 1\}$, a hard classifier is

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

The decision boundary is

$$w^\top x + b = 0.$$

5.2 Sigmoid and Logistic Regression

Logistic regression models

$$p(y = 1 \mid x) = \sigma(w^\top x + b),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Useful identities:

$$\sigma(-z) = 1 - \sigma(z), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

5.3 Binary Cross-Entropy

Let

$$\hat{y} = \sigma(w^\top x + b).$$

The Bernoulli likelihood is

$$p(y \mid x, w) = \hat{y}^y (1 - \hat{y})^{1-y}.$$

The negative log-likelihood is

$$\ell(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})].$$

For a dataset:

$$\mathcal{L}(w) = - \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)].$$

5.4 Gradient Derivation

Let

$$z_i = w^\top x_i + b, \quad \hat{y}_i = \sigma(z_i).$$

For one example,

$$\ell_i = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i).$$

A key result is

$$\frac{\partial \ell_i}{\partial z_i} = \hat{y}_i - y_i.$$

Then

$$\nabla_w \ell_i = (\hat{y}_i - y_i) x_i,$$

and for the full dataset,

$$\nabla_w \mathcal{L}(w) = X^\top (\hat{y} - y).$$

With L_2 regularization,

$$\nabla_w \mathcal{L}_{\text{reg}}(w) = X^\top (\hat{y} - y) + \lambda w.$$

6 Multiclass Classification

6.1 Linear Scores

For K classes, compute one score per class:

$$z_k = w_k^\top x + b_k.$$

Collect scores into

$$z = \begin{bmatrix} z_1 & z_2 & \cdots & z_K \end{bmatrix}^\top.$$

The predicted class is

$$\hat{y} = \arg \max_k z_k.$$

6.2 Softmax

The softmax function converts scores into probabilities:

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

Properties:

$$0 \leq \text{softmax}(z)_k \leq 1, \quad \sum_{k=1}^K \text{softmax}(z)_k = 1.$$

For numerical stability, use

$$\text{softmax}(z)_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}, \quad c = \max_j z_j.$$

6.3 Multiclass Cross-Entropy

If y is the correct class index,

$$\ell(z, y) = -\log\left(\frac{e^{z_y}}{\sum_{j=1}^K e^{z_j}}\right) = -z_y + \log\sum_{j=1}^K e^{z_j}.$$

If y is one-hot encoded,

$$\ell(\hat{p}, y) = -\sum_{k=1}^K y_k \log \hat{p}_k.$$

6.4 Softmax Cross-Entropy Gradient

The essential gradient is

$$\frac{\partial \ell}{\partial z_k} = \hat{p}_k - y_k.$$

For row-based batches:

$$X \in \mathbb{R}^{N \times D}, \quad W \in \mathbb{R}^{D \times K}, \quad Z = XW + \mathbf{1}b^\top \in \mathbb{R}^{N \times K}.$$

Let $\hat{P} = \text{softmax}(Z)$ row-wise and $Y \in \mathbb{R}^{N \times K}$ be one-hot labels. Then

$$\nabla_W \mathcal{L} = \frac{1}{N} X^\top (\hat{P} - Y),$$

and

$$\nabla_b \mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i).$$

7 Optimization

7.1 Gradient Descent

Gradient descent updates parameters by

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{(t)}),$$

where α is the learning rate.

If α is too small, learning is slow. If α is too large, training may diverge.

7.2 Batch, Stochastic, and Mini-Batch Gradient Descent

Batch gradient descent uses all examples:

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell_i(\theta).$$

Stochastic gradient descent uses one example:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} \ell_i(\theta^{(t)}).$$

Mini-batch gradient descent uses a batch B :

$$\nabla_{\theta} \mathcal{L}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta).$$

Mini-batches are the standard practical compromise.

7.3 Momentum

Momentum uses a velocity variable:

$$\begin{aligned} v^{(t+1)} &= \beta v^{(t)} + \nabla_{\theta} \mathcal{L}(\theta^{(t)}), \\ \theta^{(t+1)} &= \theta^{(t)} - \alpha v^{(t+1)}. \end{aligned}$$

Momentum accelerates movement in consistent directions and reduces oscillation.

7.4 Convexity

A function f is convex if

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

for all x, y and $\lambda \in [0, 1]$.

For differentiable f , convexity implies

$$f(y) \geq f(x) + \nabla f(x)^{\top} (y - x).$$

For twice differentiable f , convexity is equivalent to

$$\nabla^2 f(x) \succeq 0.$$

Linear regression with squared loss is convex. Logistic regression with cross-entropy is convex. Neural network training is generally non-convex.

8 Neural Networks

8.1 Motivation

Linear models can only learn linear decision boundaries in the chosen feature space. Neural networks learn nonlinear feature transformations automatically.

A neural network is a composition of layers:

$$f(x) = f_L(f_{L-1}(\cdots f_1(x))).$$

A typical layer is

$$a^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \quad h^{(l)} = \phi(a^{(l)}),$$

where $h^{(0)} = x$.

8.2 Activation Functions

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Tanh:

$$\tanh'(z) = 1 - \tanh^2(z).$$

ReLU:

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

Sigmoid and tanh can saturate, causing small gradients. ReLU is common because it is simple and helps reduce vanishing gradient problems.

8.3 Forward Pass

For a two-layer neural network:

$$\begin{aligned} a^{(1)} &= W^{(1)}x + b^{(1)}, \\ h^{(1)} &= \phi(a^{(1)}), \\ a^{(2)} &= W^{(2)}h^{(1)} + b^{(2)}, \\ \hat{y} &= g(a^{(2)}). \end{aligned}$$

The function g is often sigmoid for binary classification or softmax for multiclass classification.

8.4 Backpropagation

Backpropagation is efficient repeated use of the chain rule. Define

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial a^{(l)}}.$$

For hidden layers,

$$\delta^{(l)} = ((W^{(l+1)})^\top \delta^{(l+1)}) \odot \phi'(a^{(l)}).$$

The gradients are

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^\top,$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}.$$

For a batch, average these gradients over examples.

8.5 Output-Layer Error Signals

For sigmoid plus binary cross-entropy,

$$\delta^{(L)} = \hat{y} - y.$$

For softmax plus multiclass cross-entropy,

$$\delta^{(L)} = \hat{p} - y.$$

These simplifications are extremely important for derivations.

9 Generalization and Model Selection

9.1 Train, Validation, and Test Sets

The training set is used to fit parameters. The validation set is used to tune hyperparameters. The test set is used only for final evaluation.

Hyperparameters include learning rate, regularization strength, number of hidden units, number of layers, batch size, number of epochs, and kernel parameters.

The test set should not be used repeatedly for model selection. If it is, we overfit to the test set.

9.2 Cross-Validation

In K -fold cross-validation:

1. Split the data into K folds.
2. Train on $K - 1$ folds.
3. Validate on the remaining fold.
4. Repeat so each fold is used once as validation.
5. Average validation performance.

9.3 Underfitting and Overfitting

Underfitting means the model is too simple: high training error and high validation error. Overfitting means the model is too complex: low training error and high validation error.

Common regularization methods include L_2 regularization, L_1 regularization, early stopping, dropout, and data augmentation.

10 Probabilistic Machine Learning

10.1 Maximum Likelihood Estimation

Given model $p(x \mid \theta)$ and data $\mathcal{D} = \{x_i\}_{i=1}^N$, the likelihood is

$$L(\theta) = p(\mathcal{D} \mid \theta) = \prod_{i=1}^N p(x_i \mid \theta).$$

MLE chooses

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta).$$

Usually we maximize log-likelihood:

$$\ell(\theta) = \sum_{i=1}^N \log p(x_i \mid \theta),$$

or minimize negative log-likelihood:

$$-\ell(\theta) = -\sum_{i=1}^N \log p(x_i \mid \theta).$$

10.2 Maximum A Posteriori Estimation

MAP uses a prior:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)p(\theta).$$

MAP chooses

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} [\log p(\mathcal{D} \mid \theta) + \log p(\theta)].$$

The negative log prior acts like a regularizer.

10.3 Generative vs Discriminative Models

A generative model learns

$$p(x, y) = p(x \mid y)p(y).$$

Then it predicts with Bayes' rule:

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

A discriminative model directly learns

$$p(y \mid x)$$

or a direct decision rule.

Generative examples: Naive Bayes, Gaussian discriminant analysis, Gaussian mixture models.
Discriminative examples: logistic regression, SVMs, neural networks.

11 Naive Bayes

11.1 Conditional Independence Assumption

Naive Bayes assumes features are conditionally independent given the class:

$$p(x | y) = \prod_{j=1}^D p(x_j | y).$$

Therefore,

$$p(y | x) \propto p(y) \prod_{j=1}^D p(x_j | y).$$

Prediction uses

$$\hat{y} = \arg \max_y \left[\log p(y) + \sum_{j=1}^D \log p(x_j | y) \right].$$

The log form avoids numerical underflow.

11.2 Bernoulli Naive Bayes

For binary features $x_j \in \{0, 1\}$,

$$p(x_j | y = c) = \theta_{jc}^{x_j} (1 - \theta_{jc})^{1-x_j}.$$

The MLE is

$$\hat{\theta}_{jc} = \frac{\#\{i : y_i = c, x_{ij} = 1\}}{\#\{i : y_i = c\}}.$$

With Laplace smoothing,

$$\hat{\theta}_{jc} = \frac{\#\{i : y_i = c, x_{ij} = 1\} + \alpha}{\#\{i : y_i = c\} + 2\alpha}.$$

11.3 Multinomial Naive Bayes

For word counts,

$$p(x | y = c) \propto \prod_{j=1}^D \theta_{jc}^{x_j}.$$

The MLE is

$$\hat{\theta}_{jc} = \frac{\text{count of word } j \text{ in class } c}{\text{total word count in class } c}.$$

With Laplace smoothing,

$$\hat{\theta}_{jc} = \frac{\text{count}_{jc} + \alpha}{\sum_{j'} \text{count}_{j'c} + D\alpha}.$$

12 Gaussian Discriminant Analysis

12.1 Model

Gaussian discriminant analysis assumes

$$p(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c), \quad p(y = c) = \pi_c.$$

Prediction uses

$$\hat{y} = \arg \max_c [\log p(x \mid y = c) + \log p(y = c)].$$

12.2 Linear Discriminant Analysis

LDA assumes all classes share the same covariance:

$$\Sigma_c = \Sigma.$$

The discriminant score is

$$\delta_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c.$$

Because the score is linear in x , LDA has linear decision boundaries.

12.3 Quadratic Discriminant Analysis

QDA allows different covariances Σ_c for each class. The discriminant score is

$$\delta_c(x) = -\frac{1}{2} \log |\Sigma_c| - \frac{1}{2} (x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c) + \log \pi_c.$$

Because of the quadratic term, QDA has quadratic decision boundaries.

13 Kernel Methods

13.1 Feature Maps

A nonlinear feature map transforms inputs:

$$\phi(x) \in \mathcal{H}.$$

A linear model in feature space is

$$f(x) = w^\top \phi(x).$$

This can represent nonlinear functions in the original input space.

13.2 The Kernel Trick

A kernel computes inner products in feature space:

$$k(x, z) = \phi(x)^\top \phi(z).$$

Common kernels:

$$\begin{aligned} k(x, z) &= x^\top z && \text{linear,} \\ k(x, z) &= (x^\top z + c)^d && \text{polynomial,} \\ k(x, z) &= \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) && \text{RBF/Gaussian.} \end{aligned}$$

13.3 Kernel Ridge Regression

Ridge regression can be written in dual form. Let

$$K = XX^\top,$$

where $K_{ij} = x_i^\top x_j$. Then

$$\alpha = (K + \lambda I)^{-1}y.$$

Prediction for a new point x is

$$\hat{y} = \sum_{i=1}^N \alpha_i k(x_i, x).$$

Kernel methods express predictions as weighted combinations of similarities to training examples.

14 Support Vector Machines

14.1 Maximum Margin Classifier

For labels $y_i \in \{-1, +1\}$, a linear classifier predicts

$$\hat{y} = \text{sign}(w^\top x + b).$$

The margin constraints are

$$y_i(w^\top x_i + b) \geq 1.$$

The hard-margin SVM solves

$$\min_{w, b} \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^\top x_i + b) \geq 1.$$

Maximizing the margin is equivalent to minimizing $\|w\|^2$.

14.2 Soft-Margin SVM and Hinge Loss

For nonseparable data, introduce slack variables $\xi_i \geq 0$:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

The soft-margin objective is

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i.$$

Equivalently, use hinge loss:

$$\ell_i = \max(0, 1 - y_i(w^\top x_i + b)).$$

The objective becomes

$$\min_w \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^\top x_i + b)).$$

The hinge loss is zero when the example is correctly classified with margin at least one.

15 Unsupervised Learning

15.1 K-Means Clustering

K-means partitions data into K clusters. The objective is

$$\min_{\{c_i\}, \{\mu_k\}} \sum_{i=1}^N \|x_i - \mu_{c_i}\|^2,$$

where c_i is the cluster assignment and μ_k is the centroid.

Algorithm:

1. Initialize centroids $\mu_1, \dots, \mu_K$.
2. Assign each point to the nearest centroid:

$$c_i = \arg \min_k \|x_i - \mu_k\|^2.$$

3. Update centroids:

$$\mu_k = \frac{1}{|\{i : c_i = k\}|} \sum_{i:c_i=k} x_i.$$

4. Repeat until assignments stop changing or objective stabilizes.

K-means monotonically decreases its objective, but it may converge to a local optimum.

15.2 Gaussian Mixture Models

A Gaussian mixture model assumes

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x; \mu_k, \Sigma_k),$$

where $\pi_k \geq 0$ and $\sum_k \pi_k = 1$. Each point has a latent cluster variable $z_i \in \{1, \dots, K\}$. Unlike K-means, GMM gives soft cluster assignments.

15.3 EM Algorithm

The EM algorithm alternates between estimating latent variables and updating parameters.

E-step: compute responsibilities

$$r_{ik} = p(z_i = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}.$$

M-step: update

$$\begin{aligned} N_k &= \sum_{i=1}^N r_{ik}, \\ \mu_k &= \frac{1}{N_k} \sum_{i=1}^N r_{ik} x_i, \\ \Sigma_k &= \frac{1}{N_k} \sum_{i=1}^N r_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top, \\ \pi_k &= \frac{N_k}{N}. \end{aligned}$$

EM monotonically improves the likelihood but may converge to local optima.

16 Principal Component Analysis

16.1 Variance Maximization View

Assume centered data:

$$\frac{1}{N} \sum_{i=1}^N x_i = 0.$$

PCA finds a unit direction u maximizing projected variance:

$$\max_{\|u\|=1} \frac{1}{N} \sum_{i=1}^N (u^\top x_i)^2.$$

Let

$$S = \frac{1}{N} X^\top X.$$

Then the objective is

$$u^\top Su.$$

The solution is the top eigenvector of S .

16.2 Multiple Components and Reconstruction

The top K principal components are the top K eigenvectors of S . Let $U \in \mathbb{R}^{D \times K}$ contain these components. The low-dimensional representation is

$$z_i = U^\top x_i.$$

The reconstruction is

$$\hat{x}_i = Uz_i = UU^\top x_i.$$

PCA also solves

$$\min_U \sum_{i=1}^N \|x_i - UU^\top x_i\|^2$$

subject to $U^\top U = I$.

17 Evaluation Metrics

17.1 Regression

Mean squared error:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

Root mean squared error:

$$\text{RMSE} = \sqrt{\text{MSE}}.$$

Mean absolute error:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

17.2 Classification

Accuracy:

$$\text{Accuracy} = \frac{\text{number correct}}{\text{total number}}.$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

F1 score:

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision emphasizes avoiding false positives. Recall emphasizes avoiding false negatives.

17.3 ROC and AUC

True positive rate:

$$TPR = \frac{TP}{TP + FN}.$$

False positive rate:

$$FPR = \frac{FP}{FP + TN}.$$

The ROC curve plots TPR against FPR as the classification threshold changes. AUC measures ranking quality.

18 High-Yield Derivations

18.1 Least Squares Normal Equations

$$\mathcal{L}(w) = \frac{1}{2} \|Xw - y\|^2.$$

$$\nabla_w \mathcal{L} = X^\top (Xw - y).$$

Set to zero:

$$X^\top (Xw - y) = 0.$$

$$X^\top Xw = X^\top y.$$

$$\boxed{\hat{w} = (X^\top X)^{-1} X^\top y.}$$

18.2 Ridge Closed Form

$$\mathcal{L}(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2.$$

$$\nabla_w \mathcal{L} = X^\top (Xw - y) + \lambda w.$$

$$(X^\top X + \lambda I)w = X^\top y.$$

$$\boxed{\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y.}$$

18.3 Logistic Regression Gradient

$$\hat{y} = \sigma(w^\top x),$$

$$\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}).$$

Then

$$\frac{\partial \ell}{\partial z} = \hat{y} - y, \quad \frac{\partial z}{\partial w} = x.$$

Therefore,

$$\boxed{\nabla_w \ell = (\hat{y} - y)x.}$$

18.4 Softmax Cross-Entropy Gradient

$$\hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad \ell = -\sum_k y_k \log \hat{p}_k.$$

The key result is

$$\boxed{\frac{\partial \ell}{\partial z_k} = \hat{p}_k - y_k.}$$

18.5 Backpropagation Skeleton

For layer l :

$$a^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \quad h^{(l)} = \phi(a^{(l)}).$$

Define

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial a^{(l)}}.$$

Then

$$\delta^{(l)} = ((W^{(l+1)})^\top \delta^{(l+1)}) \odot \phi'(a^{(l)}),$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}.$$

19 Exam-Ready Concept Comparisons

19.1 Loss vs Likelihood

Many losses are negative log-likelihoods. Squared error corresponds to Gaussian likelihood. Binary cross-entropy corresponds to Bernoulli likelihood. Multiclass cross-entropy corresponds to categorical likelihood. Regularization corresponds to a prior in MAP estimation.

19.2 Linear vs Nonlinear Models

A model can be linear in parameters but nonlinear in inputs. For example,

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}$$

and

$$f(x) = w^\top \phi(x)$$

is nonlinear in x but linear in w . Neural networks learn nonlinear features automatically.

19.3 Closed Form vs Iterative Optimization

Closed-form solutions: ordinary least squares and ridge regression. Iterative optimization: logistic regression, neural networks, lasso, SVMs, GMMs, and most large-scale models.

19.4 Generative vs Discriminative

Generative models learn $p(x, y)$ or $p(x | y)p(y)$. Discriminative models learn $p(y | x)$ or a direct decision boundary. Generative models can be useful with small data, missing data, or sampling. Discriminative models often perform better for pure prediction.

19.5 Why Validation Data Matters

Training loss usually decreases as model complexity increases. Validation loss estimates whether the model generalizes. The best model is usually the one with the lowest validation loss, not necessarily the lowest training loss.

20 Compact Formula Sheet

Linear Regression

$$\begin{aligned} \hat{y} &= Xw \\ \mathcal{L}(w) &= \frac{1}{2} \|Xw - y\|^2 \\ \nabla_w \mathcal{L} &= X^\top (Xw - y) \\ \hat{w} &= (X^\top X)^{-1} X^\top y \end{aligned}$$

Ridge

$$\mathcal{L}(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

$$\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$$

Logistic Regression

$$\hat{y} = \sigma(w^\top x)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\nabla_w \ell = (\hat{y} - y)x$$

Softmax

$$\hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$$\ell = - \sum_k y_k \log \hat{p}_k$$

$$\frac{\partial \ell}{\partial z_k} = \hat{p}_k - y_k$$

Gradient Descent

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$

PCA

$$S = \frac{1}{N} X^{\top} X$$

$$u_1 = \text{top eigenvector of } S$$

K-Means

$$c_i = \arg \min_k \|x_i - \mu_k\|^2$$

$$\mu_k = \frac{1}{|\{i : c_i = k\}|} \sum_{i:c_i=k} x_i$$

21 Recommended Relearning Order

For relearning CSC311 efficiently, use the following order:

1. Linear regression and least squares.
2. Probability review: likelihood, log-likelihood, MLE, MAP.
3. Logistic regression and cross-entropy.
4. Gradient descent and optimization.
5. Regularization and bias-variance.
6. Neural networks and backpropagation.
7. Generative models: Naive Bayes and GDA.
8. Kernel methods and SVMs.
9. Unsupervised learning: K-means, GMM, EM, PCA.
10. Evaluation and model selection.

For proof- and derivation-heavy exams, prioritize:

1. Deriving gradients.
2. Explaining likelihood-to-loss connections.
3. Tracking matrix shapes.
4. Bias-variance reasoning.
5. Regularization effects.
6. Backpropagation.
7. Comparing models conceptually.

Final summary: machine learning is not just a collection of algorithms. Each method should be understood through four questions: What is the model? What loss does it minimize? How is it optimized? Why should it generalize?